Merge Sort

An optimal sorting algorithm based on Divide & Conquer approach ($\Theta(n \log n)$ time)

Divide & Conquer

Many useful algorithms are recursive in structure

Divide the instance into a number of subinstances that are smaller

Conquer the subinstances by solving them recursively (or solve them directly if small enough)

Combine the solutions of the subinstances into the solution of the original instance

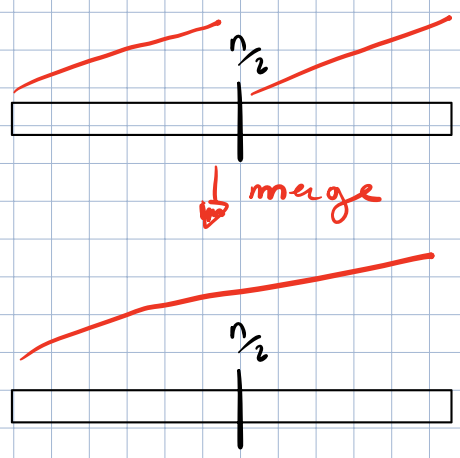
Usually the Conquer step is easy and cheap and only one of the two (Divide and Combine) is expensive.

Merge Sort

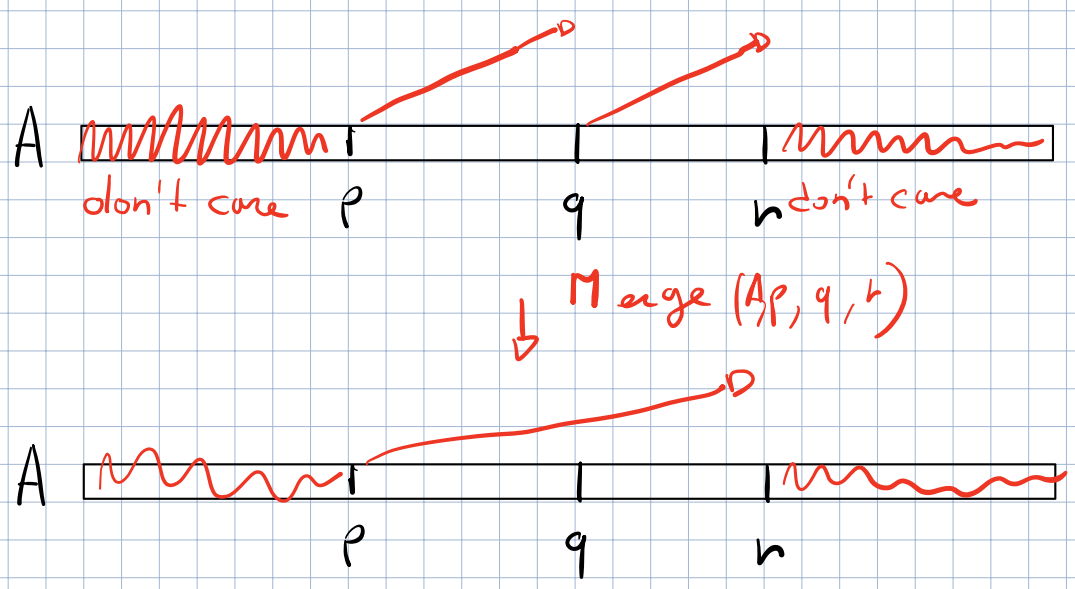
Divide an n-element array into <u>two</u> subarrays of size $\approx \frac{n}{2}$

Conquer sorts two subarrays recursively.
if $n \leq 1$, stop

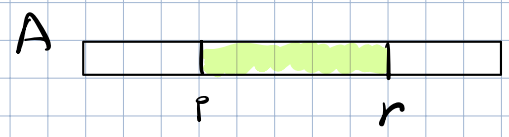
Combine merges the two sorted subarrays to produce their sorted union



Merge (A, p, q, r)



MergeSort (A, p, r)

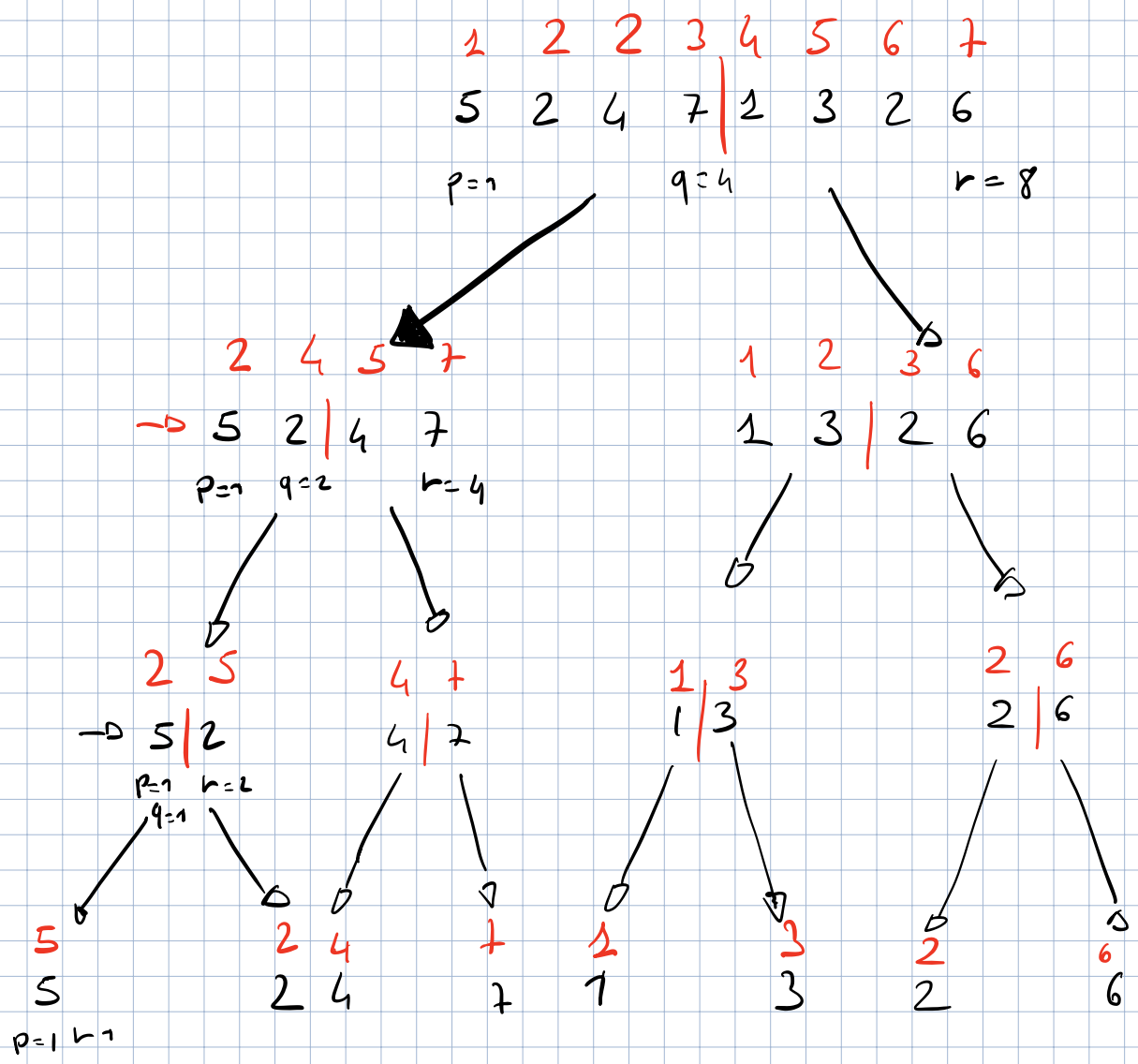


if p < r :

$q = \lfloor \frac{p+r}{2} \rfloor$ // Divide

MergeSort (A, p, q)
MergeSort (A, q+1, r) // Conquer

Merge (A, p, q, r) // Combine

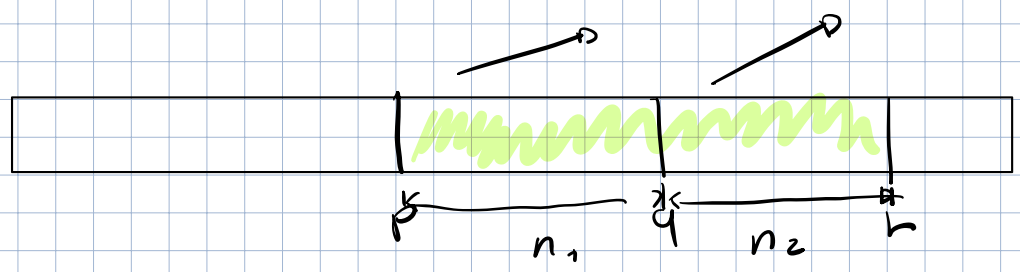


2 4 5 7 $+\infty$ 1 2 3 6 $+\infty$

1 2 2 3 4 5 6 7

LINEAR TIME

Merge (A, p, q, r)



$n_1 = q - p + 1$

$n_2 = r - q$ // $O(1)$

let $L[1, \dots, n_1+1]$ and

$R[1, \dots, n_2+1]$ be two new arrays // $O(1)$

for $i = 1$ to n_1

$L[i] = A[p+i-1]$ // $\Theta(n_1)$

$L[n_1+1] = +\infty$ // sentinel

```
for i = 1 to n2
    R[i] = A[q+i]
R[n2+1] = +∞        // sentinel
```

// $\Theta(n_2)$

```
i = j = 1

for k = p to r
    if L[i] ≤ R[j]
        A[k] = L[i]
        i += 1
    else
        A[k] = R[j]
        j += 1
```

$\Theta(n_1+n_2)$

LINEAR IN THE SIZE OF THE SUBARRAY

IS MS STABLE? YES

IS MS INPLACE? NO

can we live without sentinels?

yes with three loops!!!

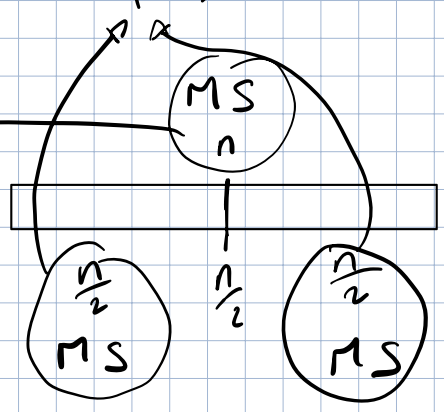
Merge Sort Analysis

Are best and worst cases different? NO

$$T(n) = \begin{cases} O(1) & \text{if } n \le c \\ 2T\left(\frac{n}{2}\right) + O(1) + \Theta(n) \end{cases}$$

divide

merge



Write it in Python (with CHATGPT)

```
def solve(n):
```

$$T(n) = \begin{cases} O(1) & \text{if } n \le c \\ 2T\left(\frac{n}{2}\right) + \cancel{O(1)} + \Theta(n) \end{cases}$$

General Recurrence for D&C approach

$$T(n) = \begin{cases} O(1) & \text{if } n < c \\ a\,T\left(\frac{n}{b}\right) + D(n) + C(n) \end{cases}$$

a = # of sub instances $\qquad \frac{n}{b}$ = size of the subinstances

$D(n)$ = function to account for the cost of Divide step

$C(n)$ = function to account for the cost of Combine Step

Merge Sort

$a = 2$ $\qquad D(n) = O(1)$

$b = \frac{n}{2}$ $\qquad C(n) = \Theta(n)$

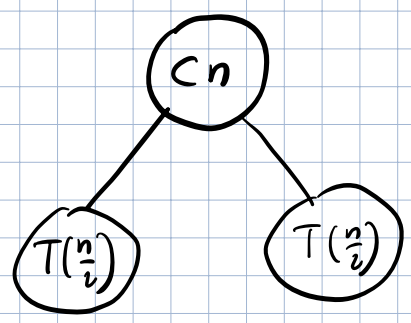
$$T(n) = \begin{cases} O(1) & \text{if } n \leq c \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \end{cases}$$

Solve recurrences with recursion tree

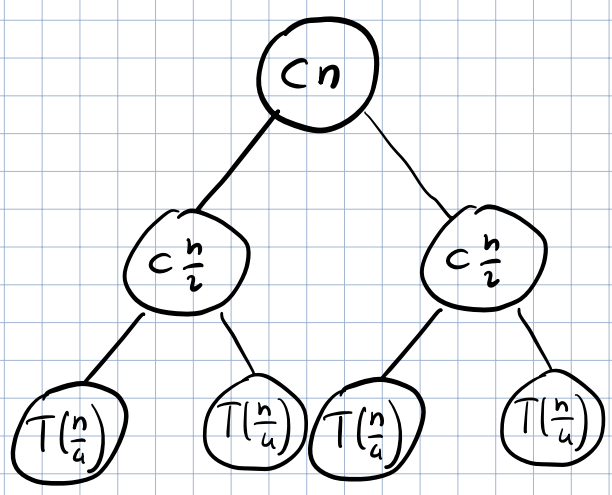
$$T(n) = \begin{cases} c & \text{if } n \le 1 \\ 2T\left(\frac{n}{2}\right) + cn & \left(\Theta(n)\right) \end{cases}$$

1

2

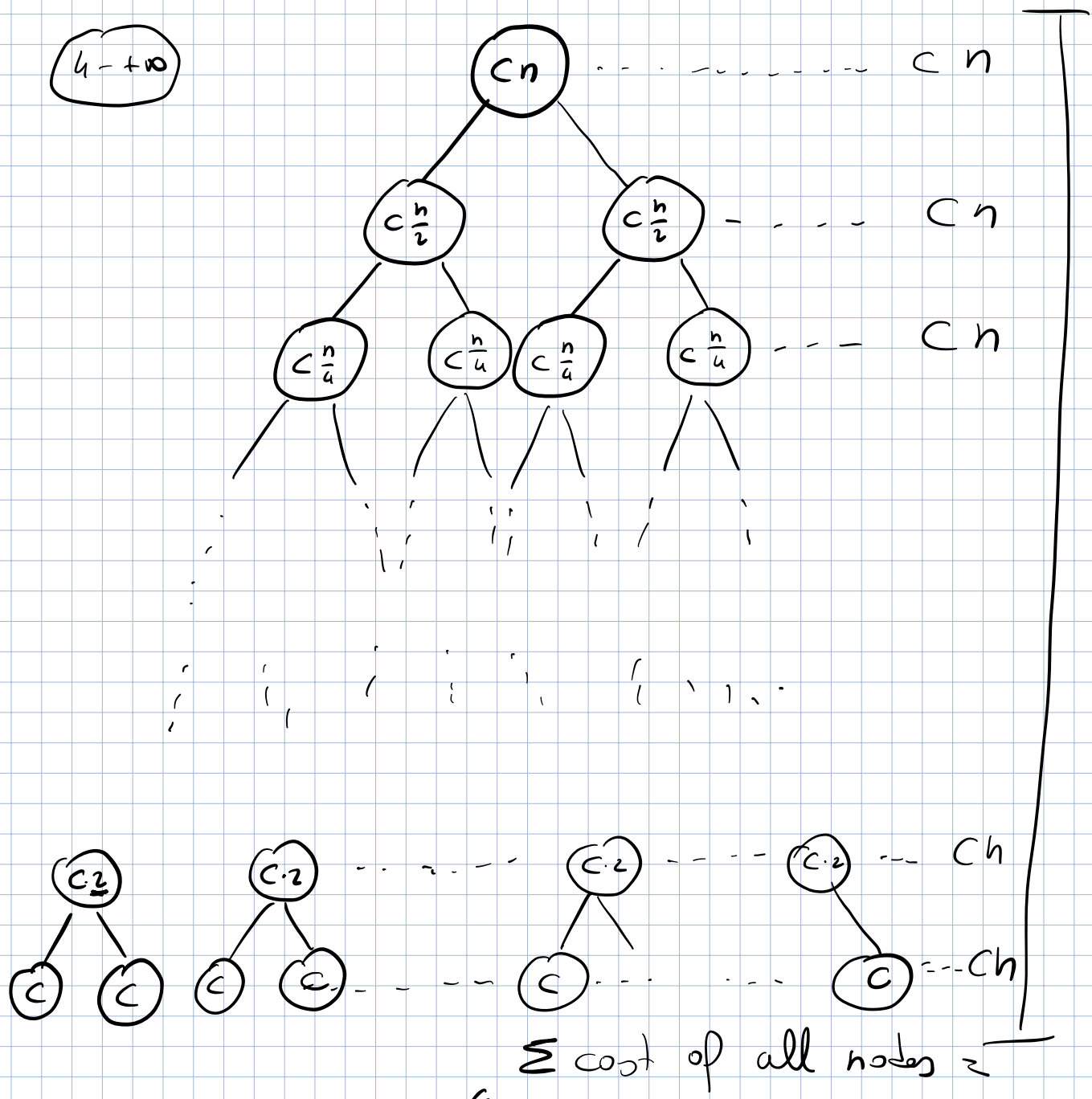


3



$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + c\frac{n}{2}$$

4 - +∞



Σ cost of all nodes =

$$T(n) = \# \text{levels} \cdot c\,n$$

levels = # times divide n by 2 to get 1

$$T(n) = c \cdot n \cdot \log_2 n = \Theta(n \log n)$$

$n = 4\ B$

$n^2 = n \cdot n$ 4B

$n \lg n = n \cdot \log n$ 32

n^3

n^2 $n\sqrt{n}$

$n \log n$

n

$\log n$

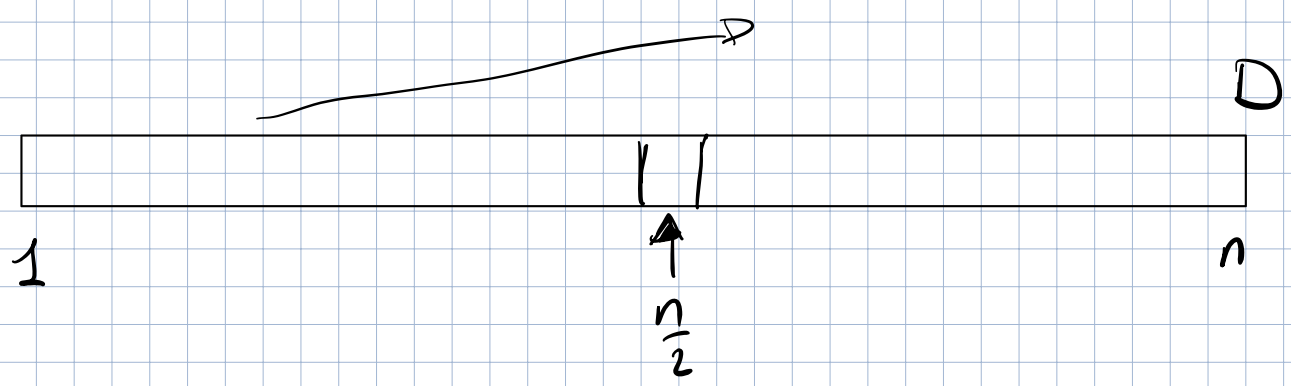
$O(1)$

```
for e in L:
    if e in D:
```

Search a key

Given a dataset D which is sorted

Search for a key k in D



if $k < D[\frac{n}{2}]$

search on the left

else

search on the right

```
BinarySearch(A, k, p, r)
    if p > r  return False
    if p == r
        return A[p] == k
    q = ⌈(p+r)/2⌉
    if A[q] == k  return True
    if k < A[q]
        return BinarySearch(A, k, p, q-1)
    else
        return BinarySearch(A, k, q+1, r)
```

Recurrence

$$T(n) = \begin{cases} O(1) & \text{if } n \leq 1 \\ a\,T\left(\frac{n}{b}\right) + D(n) + C(n) \\ = 1\,T\left(\frac{n}{2}\right) + O(1) \end{cases}$$

Solve

$$T(n) = \begin{cases} O(1) & \text{if } n \le 1 \\ 2T(\frac{n}{2}) + \Theta(n) \end{cases}$$ in python

```
def solve(n):
    if n ≤ 1 : return 10
    return 2 * solve(n // 2) + 10 n
```

CHATGPT claims its exponential time

$$T'(n) = \begin{cases} O(1) & \text{if } n \le 1 \\ 1 \cdot T'(\frac{n}{2}) + O(1) \end{cases}$$

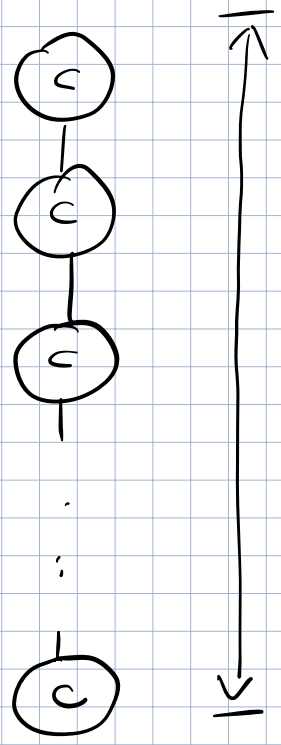
running time of solve function

1

2

3

$4 - +\infty$



levels
$= \log n$

$$T'(n) = c \log n = \Theta(\log n)$$

$n = 32$

$2^n \rightarrow 4\ B$

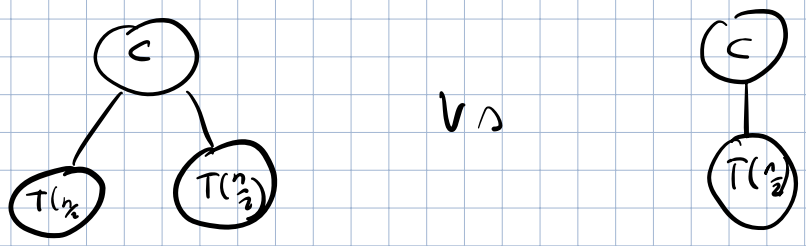
$\log n \rightarrow 5$

```
def solve(n):
    if n ≤ 1 : return 10
    return solve(n//2) + solve(n//2) + 10 n
```

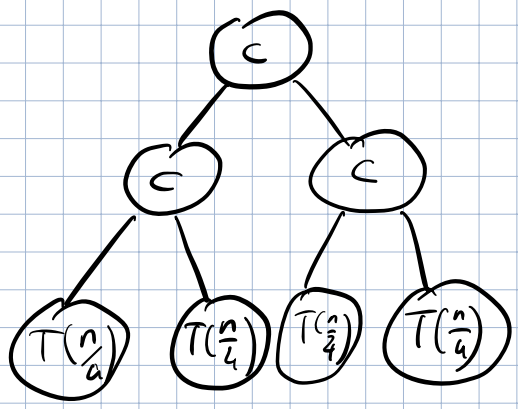
$$T(n) = \begin{cases} O(1) & \text{if } n \le 1 \\ 2 \cdot T'\left(\frac{n}{2}\right) + O(1) \end{cases}$$

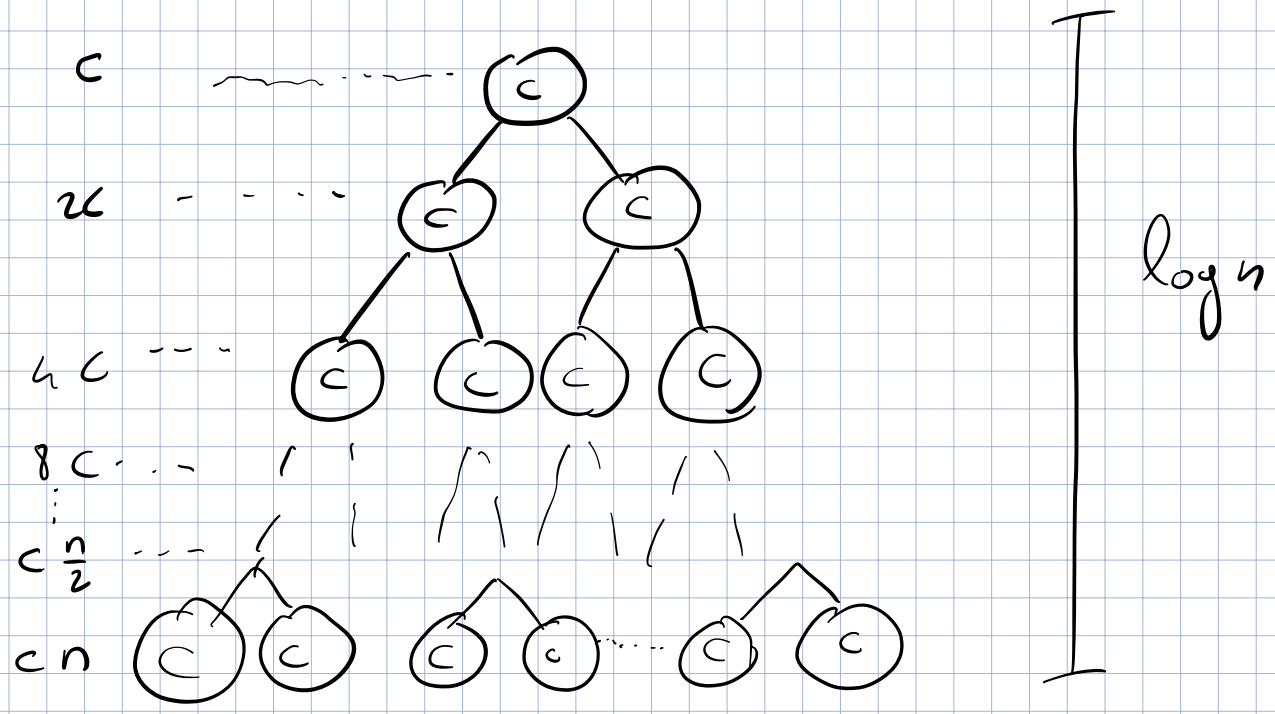
1

2



3





$$T(n) = c\sum_{k=0}^{\log n} 2^k \leq c2n = \Theta(n)$$

solve $T(n) = \begin{cases} O(1) & \text{if } n \leq c \\ T(\frac{n}{2}) + \Theta(n) \end{cases}$

write a recursive function with this complexity